

THE JOURNEY OF AUTORAG

From AutoML for RAG to Search Infra for Agents

How a RAG auto-optimization tool with 4.8K stars became a mission to build the knowledge-base layer & the all-purpose librarian search agent that AI agents are missing.

Jeffrey Kim · Creator of AutoRAG

NomaDamas — AI Hacker House, Seoul

Marker Inc.

WHOAMI

Hi, I build retrieval systems.

I created **AutoRAG** and work out of **NomaDamas**, an AI hacker house in Seoul. I spend my time on one question:

"Which retrieval pipeline is **actually** best — for **this** data, **this** use case?"

~/nomadamas

```
# open-source I ship & maintain
AutoRAG ★ 4.8k RAG AutoML
AutoRAG-Research ★ 144 RAG bench
MinSync ★ 51 incremental index
agentdir ★ 51 virtual file tree
jikji ★ 35 agent file maps

# awards
Minister of Science & ICT Award
```

THE MAP FOR THE NEXT 20 MINUTES

One product, three eras — the goal kept changing.

2024 →

AutoRAG

Find the optimal RAG pipeline for **your data**, automatically.

GOAL · AutoML for RAG



2025 →

AutoRAG-Research

Unify datasets, SOTA pipelines & metrics. Make RAG research **reproducible**.

GOAL · Research automation

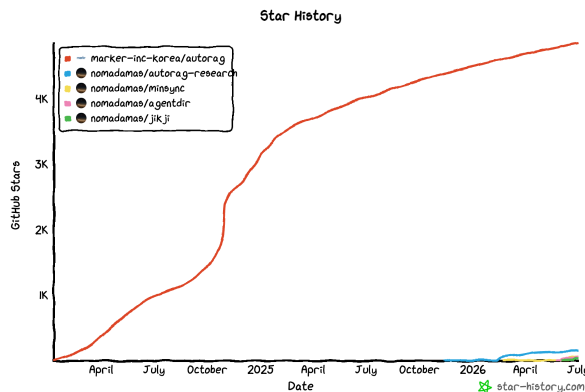


2026 →

AutoRAG 2.0

The **knowledge-base layer** & librarian search agent for the agent era.

GOAL · Search infra for agents



GitHub star history · star-history.com — AutoRAG (red) to ~4.8K; the new agent-era repos rising bottom-right.

2024 · HOW IT STARTED

There were already too many frameworks.

RAG frameworks? Dozens. Evaluation frameworks? Plenty. Yet everyone kept asking the same unanswered question:

"There are many RAG pipelines & modules out there — but you **don't know which one is best for your own data and use case.** Building and evaluating all of them is painfully slow."

The missing idea wasn't another pipeline. It was the concept from a neighboring field:

Hyperparameter tuning for ML

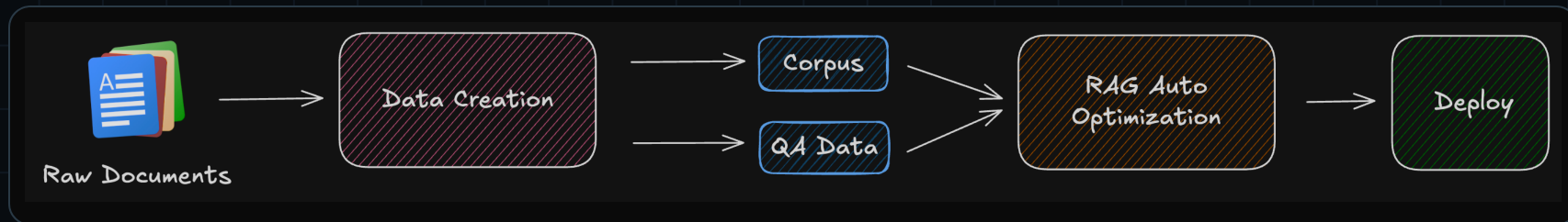
AutoML

▼ applied to retrieval ▼

AutoRAG

Search the space of parsers · chunkers · retrievers · rerankers · prompts · LLMs
— let evaluation pick the winner.

AUTORAG · AUTOML FOR RAG



★ **4.8K**

GitHub stars · 400+ forks

Trendshift

Weekly-trending repository

arXiv

2410.20878 · published

**Uber · SKT
AWS · Hanwha**

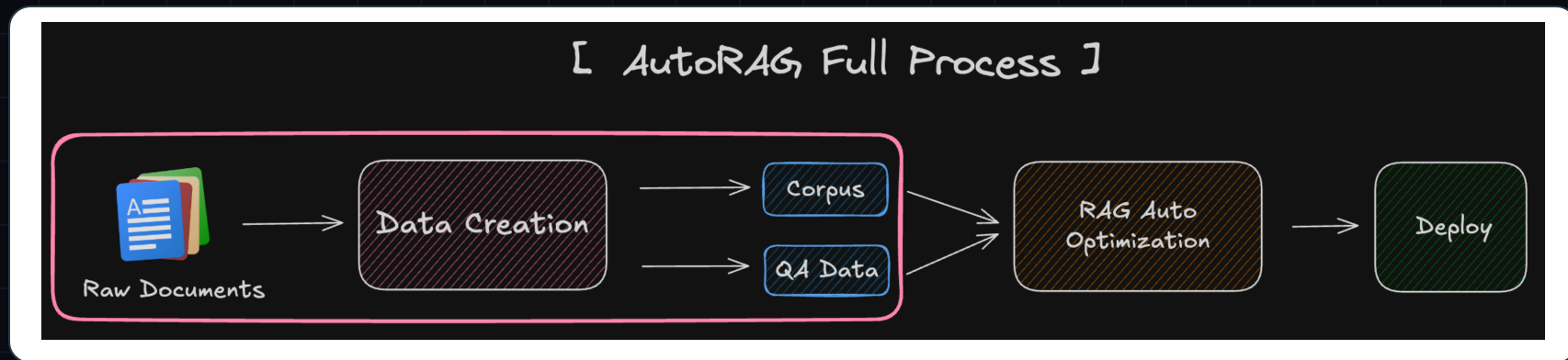
Real-world adopters

Give AutoRAG your QA + corpus data → it evaluates many RAG module combinations automatically → returns the best pipeline for **your** use case, ready to deploy.

AUTORAG · STEP 0 — YOU NEED DATA TO OPTIMIZE ON

Optimization needs a QA set + a corpus.

So AutoRAG ships a full data-creation pipeline: **parse** → **chunk** → **QA generation** — turn raw PDFs/docs into an evaluation dataset.



parse · pdfminer / llama / clova

chunk · token / semantic

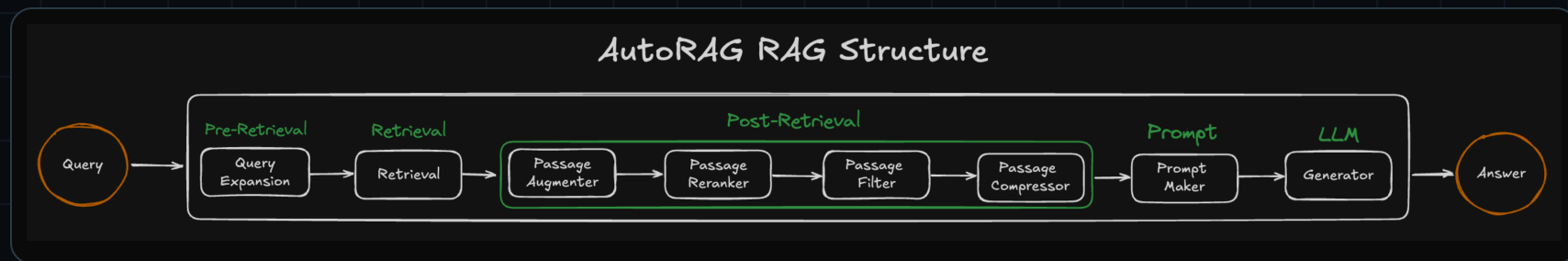
QA · factoid · evolve · filter

→ qa.parquet + corpus.parquet

AUTORAG · HOW IT OPTIMIZES

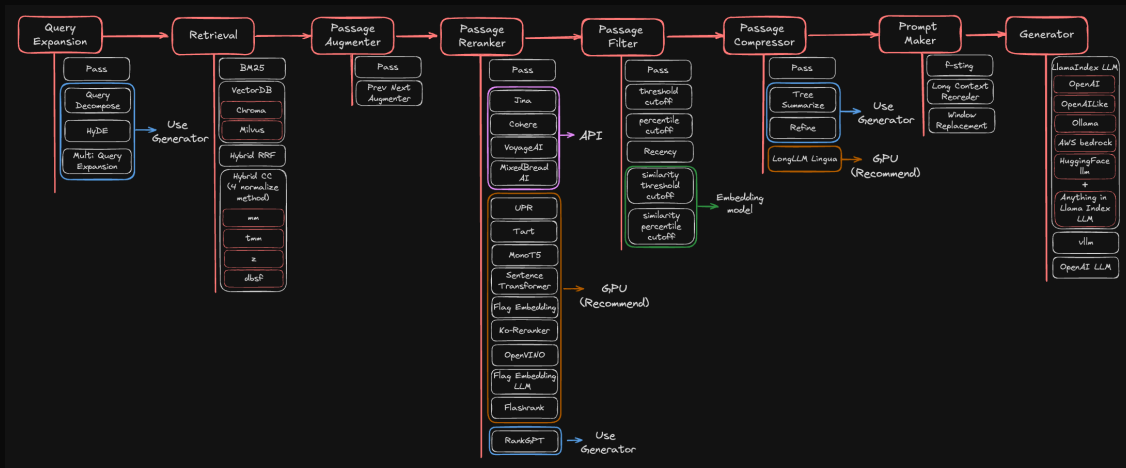
A RAG pipeline = a chain of nodes.

AutoRAG models the pipeline as ordered node lines. Each node has swappable modules; the framework greedily searches for the best module + params at every stage.



Query → Pre-Retrieval → Retrieval → Post-Retrieval → Prompt → LLM → Answer. Every box is a decision point AutoRAG optimizes.

Every node has many modules — AutoRAG tries them for you.



Configure the space in one YAML. AutoRAG runs the trials, evaluates with retrieval + generation metrics (F1, Recall, nDCG, MRR, METEOR, ROUGE, SemScore...), and emits a **summary.csv** with the winning pipeline.

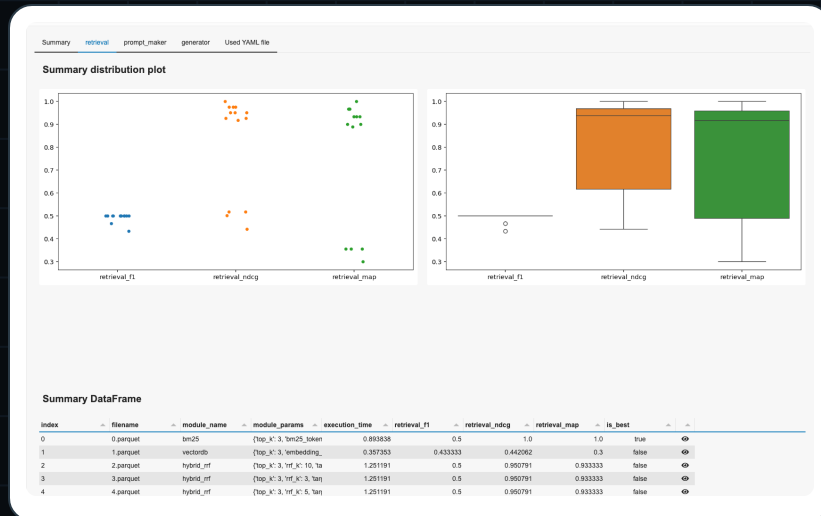
Optimize → inspect → deploy in one flow.

- Interactive **dashboard** to compare every trial & module
- Deploy the winner as **code**, an **API server**, or a **web app**
- YAML in, best-pipeline artifact out — fully reproducible

autorag evaluate

autorag dashboard

autorag run_api



AutoRAG optimization dashboard

2025 · WHERE AUTORAG HIT ITS CEILING

Then the ground shifted under RAG.

CONSTRAINT 01

Locked to "advanced RAG"

Agentic RAG arrived. AutoRAG's fixed node structure couldn't express the wild variety of new pipelines.

CONSTRAINT 02

Datasets are hard

Every benchmark has a different format. Building & embedding evaluation data is slow, repetitive, error-prone.

CONSTRAINT 03

Research won't reproduce

Every paper claims SOTA. Re-implementing each one to compare fairly is a research project by itself.

A tool that optimizes **one** pipeline shape can't answer "**which pipeline shape is even right?**"

AutoRAG-Research

The Standard for RAG Benchmarking

	MMLU		NQ		HotpotQA		RahoMQA		MBAG		WalGA		InfoSeek		LYBeeth		VetooRAG		NIM		Video-Yrath			Add Dataset +
Models	Acc	ESI	F1	EM	F1	EM	PI	Acc	R-L	BEST	Acc	R-L	BEST	Acc	R-L	BERT	BEST	R-L	BERT	BEST	R-L	BEST		
Kolve	74.39	39.59	28.86	31.16	29.53	2.80	7.80	48.22	56.12	91.76	18.10	28.83	19.78	36.31	90.76	33.40	-	-	-	-	-	-	-	-
ParagraphRAG	74.39	39.25	31.32	31.38	31.43	5.13	9.21	42.28	51.70	91.70	19.75	24.01	17.02	32.84	99.97	37.76	-	-	-	-	-	-	-	-
DocumentRAG	71.39	21.95	31.96	31.35	34.43	5.73	7.40	43.88	43.87	91.70	16.50	15.73	16.34	85.60	89.27	37.38	-	-	-	-	-	-	-	-
TaMeRAG	72.39	11.50	16.73	18.43	33.28	3.65	13.86	43.88	41.57	91.10	16.50	16.47	16.34	84.04	80.43	37.45	-	-	-	-	-	-	-	-
langerRAG	72.31	17.15	21.97	16.45	36.18	2.30	3.86	52.33	44.75	91.47	9.15	16.47	12.16	83.06	96.77	28.45	-	-	-	-	-	-	-	-
ChERAG	73.33	17.15	23.37	19.30	36.15	2.30	6.80	52.19	63.96	99.98	19.13	31.73	31.10	37.87	91.47	33.50	-	-	-	-	-	-	-	-
langerRAG	74.91	16.70	23.66	19.30	36.35	3.23	4.15	52.58	63.99	94.93	9.83	31.72	31.10	37.67	91.47	31.31	-	-	-	-	-	-	-	-
ChERAG	74.91	19.91	23.60	26.00	20.50	2.50	5.78	48.94	64.57	94.97	11.25	33.85	30.59	85.05	91.54	31.91	-	-	-	-	-	-	-	-
ParagraphRAG	70.86	19.30	20.71	15.33	26.93	3.69	7.59	40.43	55.30	92.90	19.05	23.65	18.61	32.41	30.68	32.02	-	-	-	-	-	-	-	-
DocumentRAG	74.39	20.97	21.01	18.56	36.93	3.80	8.80	44.43	43.63	91.20	19.10	21.88	18.01	31.03	99.88	33.86	-	-	-	-	-	-	-	-
TaMeRAG	72.39	29.91	22.05	18.30	26.63	3.08	8.70	44.13	48.83	91.30	19.10	21.88	16.03	33.75	80.70	33.87	-	-	-	-	-	-	-	-
langerRAG	73.91	79.97	20.05	23.30	22.45	3.89	8.54	46.73	58.79	91.27	19.10	23.04	15.03	85.75	90.19	31.31	-	-	-	-	-	-	-	-
ChGRAG	74.91	79.97	21.97	23.00	23.33	6.33	8.47	46.70	53.35	93.31	17.05	23.25	85.91	31.07	90.19	31.31	-	-	-	-	-	-	-	-
Arg	74.39	39.65	28.86	30.03	23.81	3.30	7.78	63.81	54.73	94.71	19.05	29.05	17.89	34.24	30.33	34.67	-	-	-	-	-	-	-	-
-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
-	-	-	-	-	-	-	-	-	-	-	-	-	-	-										

One framework that unifies **datasets**, **SOTA pipelines**, and **metrics** — so you can benchmark your idea against the real state of the art with one command.

"Every dataset differs" → unified & pre-embedded

"Every paper claims SOTA" → run them all, compare

AUTORAG-RESEARCH · ONE POSTGRESQL, EVERYTHING PRE-BUILT

DATASETS

Unified & pre-embedded

BEIR · MTEB · RAGBench · MrTyDi · BRIGHT · ViDoRe
v1-v3 · VisRAG · Open-RAGBench. Text **and** image.

PIPELINES

SOTA from papers

DPR · BM25 · HyDE · Query Rewrite · Hybrid RRF/CC ·
BasicRAG · IRCot · ET2RAG · VisRAG · MAIN-RAG.

METRICS

Retrieval + generation

Recall · Precision · F1 · nDCG · MRR · MAP · BLEU ·
METEOR · ROUGE · BERTScore · SemScore.

benchmark in 3 commands

```
$ autorag-research ingest --name beir --extra dataset-name=scifact
$ autorag-research data restore beir beir_arguana_test_qwen # pre-computed embeddings
$ autorag-research run --db-name=beir_scifact_test # all pipelines, one leaderboard
```


Plugins + agent skills = experiment at agent speed.

- **Plugin system** — scaffold a plugin, `pip install -e .`, run beside the built-ins. No fork.
- **Agent skill** — your coding agent queries results straight from PostgreSQL in natural language.
- Spin up dozens of experiments & ask questions conversationally.

```
npx skills add NomaDamas/AutoRAG-Research
```

autorag-query · agent skill

You ► Which pipeline has the best BLEU score?

Agent ► `hybrid_search_v2` achieved the highest BLEU score of `0.85`.

queried directly from your results DB

2026 • AND THEN...

The agent era arrived — for real this time.

Claude Code. Codex. Autonomous coding & knowledge agents everywhere.

So we asked ourselves the only question that mattered:

In **this** era, what should AutoRAG become?

Build the missing knowledge-base layer for agents.

Agents are brilliant at reasoning and tool use — but the layer that **organically integrates & manages RAG and many data sources**, and retrieves information from **everywhere** (including your local files), is still missing.

AutoRAG 2.0 fills that gap — built with **NIPA** open-source program support.

CONSORTIUM

Lead

Marker Inc. × NomaDamas

Core architecture · module development · OSS governance

Partner

BrainCrew — TeddyNote

LangChain Ambassador · agent validation · education

Partner

2e Consulting

Public & finance domain requirements · pilot validation

Supported by NIPA — 2026 Open-Source AI-SM program

WHY A KNOWLEDGE-BASE LAYER?

Right now, org knowledge is flyers on a fridge.

TODAY

Flyers on the fridge

- Dig through a pile to find the menu you want
- Call each restaurant to check what's available
- Closed shops still have flyers up — stale & noisy

WITH AUTORAG 2.0

A delivery app

- Search the menu you want, instantly
- Prices, options, reviews — categorized
- Closed shops auto-removed — always current

An agent-native knowledge-base layer — the middle layer that lets an agent use your documents at 200%.

A 5-layer knowledge-base layer between your files and the agent.

▲ Upper agent — Claude Code · Codex · OpenCLAW ▲

05 **Operations**
self-re-evaluating optimization loop

04 **Agent Interface**
CLI & Skills — agents call local knowledge

03 **Search Orchestration**
hybrid semantic + keyword + metadata

02 **Storage**
vector DB + virtual FS · incremental indexing

01 **Document Understanding**
open-source parsers · PDF · HWP · DOCX

▼ Local document folder ▼

- L01 Unstructured parsing**
Parse tables, images & formulas into logical structure.
- L02 Integrity & freshness**
Detect file changes in real time; re-index only the delta.
- L03 Intelligent hybrid search**
Combine methods dynamically to find the best evidence.
- L04 Instant agent connection**
No bespoke integration — CLI + Skills, ready to call.
- L05 Sustained advantage**
Learn from usage logs; keep tuning the strategy.

Each layer is a swappable module — not locked to any vector DB, engine, or model.

THE EVOLUTION OF "AUTO"

Naive optimization → memory-based personalization.

AutoRAG 1.0

One-shot search over a config space

You bring a static eval set. AutoRAG finds one "best" pipeline. Great — until the data, users, and questions drift.

AutoRAG 2.0

A memory-driven, personalizing loop

Operation logs → the system re-evaluates workflows, adapts retrieval to **your** corpus, queries, feedback. Optimization becomes **personalization**.

grid search

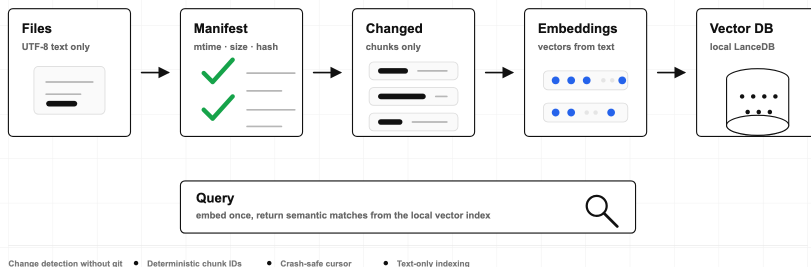


memory + feedback + self-tuning

THE BUILDING BLOCKS • BEING INTEGRATED INTO AUTORAG

MinSync

Manifest-based incremental vector indexing, no git required.



MinSync • Rust • *51

Incremental indexing

Git-free change detection via mtime/size/hash. Re-embed only changed chunks, sweep stale ones — the Storage layer stays fresh at near-zero cost.

agentdir • Rust • *51

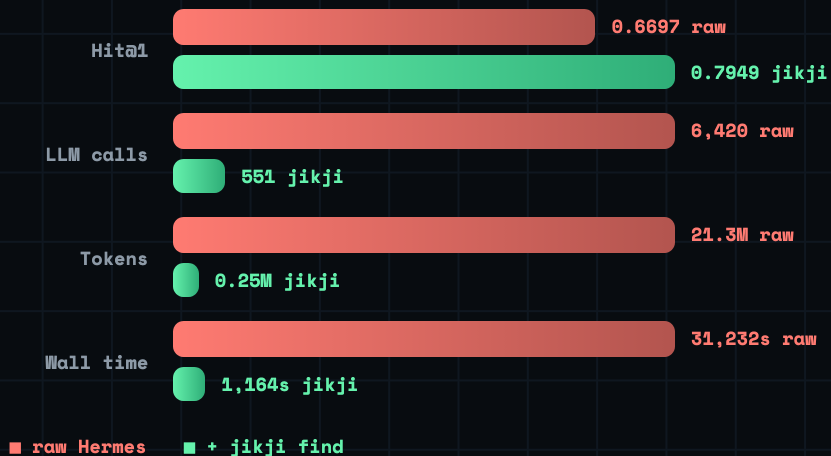
Virtual directories

Present the same originals in a purpose-built, read-only layout. CoW/reflink — no data copy. Give agents a better working tree without moving files.

JIKJI · A SIMPLE FILE MAP, A HUGE WIN

Prepare a local map first → agents search less, hit more.

HippoCamp benchmark · 551 local file-search cases · raw Hermes agent vs. the same agent with **jikji find**.



86×

fewer total tokens

11.7×

fewer LLM calls

+12.5pt

higher Hit@1 accuracy

Metadata + file maps + parser caches + graph routes → one ranked candidate slate. The agent judges once instead of crawling.

WHERE THE JOURNEY IS HEADING

The all-purpose librarian search agent.

One agent that **manages many data sources**, **digs everywhere** for information — local files included — and **keeps itself current** through memory-based personalization.

AutoML for RAG



Research automation



Search infra for agents

github.com/Marker-Inc-Korea/AutoRAG

NomaDamas · MinSync · agentdir · jikji

Thank you. Let's build the knowledge layer agents deserve.